LAB WALKTHROUGH

homa-lab

Testing the Linux kernel module that implements the Homa transport protocol: four hosts running `homa.ko`, a switch with real priority queues, and experiments that show what Homa does on the wire, under incast, under loss — and where its defaults stop fitting.

Version 0.3.0 · 15 October 2026 · 30 of 30 tests passing
Lab directory `~/homa-lab` (local git repository) · Lab Hub card `homa-lab`
Kernel `6.17.8-061708-generic` · HomaModule `main @ 65cecd7` (2026-10-08)

Contents

1. What the lab is, and why
2. Homa for dummies
3. Homa in five minutes
4. Architecture
5. How the image is built
6. Getting started
7. Walkthrough A — the lab comes up
8. Walkthrough B — one RPC on the wire
9. Walkthrough C — sw1's priority queues
10. Walkthrough D — Homa against TCP
11. Walkthrough E — incast
12. Walkthrough F — loss recovery
13. Walkthrough G — the resend timers
14. Walkthrough H — segmentation offload
15. Configuration and metrics
16. Tests
17. Findings
18. Lessons learned building it
19. Troubleshooting
20. What is not built yet

Appendix A — reference: lab.sh

Appendix B — sw-qos

Appendix C — lab.conf and layout

1. What the lab is, and why

Homa is a datacenter transport protocol designed by John Ousterhout's group at Stanford as an alternative to TCP for remote procedure calls. Its claim is low *tail* latency for short messages under load — 10 to 100 times lower than TCP in their measurements — obtained by being message-based instead of byte-stream based, by letting *receivers* schedule incoming traffic, and by using the priority queues already present in datacenter switches.

Homa is implemented as an out-of-tree Linux kernel module, PlatformLab/HomaModule. Upstreaming into the Linux kernel started in October 2024 and proceeds in stages; it is not in a mainline release yet. This lab tests that module.

> **What this lab is for.** Behaviour and correctness, not performance. The hosts are KVM guests on a single 12-core machine, so absolute latencies and throughputs are not datacenter numbers. What the lab can show honestly is: the build works and passes its own tests; Homa works end to end over IPv4 and IPv6; what it puts on the wire; how it compares *on the same machines* with TCP; what priority queues do for it under incast; how it recovers from loss; and how its defaults behave on a slower network than it was tuned for.

- **Pinned and reproducible.** The kernel version and the HomaModule commit are fixed in `lab.conf`; one command builds the image from nothing, compiles the module and runs its 840 unit tests.
- **A switch that matters.** Homa depends on switch priority queues. sw1 is a Linux VM whose ports implement exactly what Homa's documentation asks of a top-of-rack switch: strict priority queues selected by the top three bits of DSCP. They can be swapped for a single FIFO to see the difference.
- **Measured with Homa's own tools.** `homa_test`, the echo `server` and `cp_node` come from HomaModule's `util/`; the lab adds orchestration (`tools/homalab.py`), a packet dissector, and Robot Framework suites that turn each experiment into checks.

2. Homa for dummies

No networking background needed. If you already know how TCP works, skip to section 3.

The problem: computers in a datacenter talk in small requests

When you load a web page, search, or open an app, the service behind it is not one computer. One request fans out to dozens or hundreds of servers, each asked a small question ("what's in this user's cart?", "give me result 7 to 12"), and the page is ready only when the **slowest** answer arrives. So what matters is not the average speed of a message but the speed of the unlucky ones: the 99th percentile, the **tail**. If 1 message in 100 is slow and a page needs 100 of them, most pages are slow.

Almost all of these messages are tiny — a few hundred bytes — mixed with occasional big transfers (a file, a backup, a model). The problem is the mix: the tiny messages get stuck behind the big ones.

What TCP does, in one paragraph

TCP, the protocol nearly everything uses, was designed in the 1970s–80s for the internet. It opens a **connection** between two computers (a phone call), then sends an endless **stream of bytes** down it, in order. It doesn't know where one message ends and the next begins. Each sender guesses how fast it may go by speeding up until packets get lost, then backing off. The network's queues fill up before anyone slows down, and a small message that arrives behind a big transfer waits its turn in the queue.

> **An analogy.** TCP is a single-lane road where every truck driver accelerates until there is a crash, then brakes. A pizza scooter that happens to be behind a convoy of trucks waits behind all of them.
>
> Homa is a loading dock with a dispatcher. Small parcels may drive straight in, in an express lane. Anyone with a truckload phones ahead; the dispatcher, who sees everything heading for the dock, tells each truck when to come — the one with the least left to unload goes first. Nobody crashes into a queue to find out it is full, and the scooter never waits behind a truck.

How Homa works, in five ideas

1. **Messages, not streams.** You send a whole message and the other side receives a whole message. A request plus its answer is a **remote procedure call (RPC)** — exactly what datacenter software does all day.
2. **No connections.** No call to set up and hang up. One Homa socket can talk to thousands of other machines with no per-machine setup, where TCP needs a separate connection to each.
3. **Small messages just go.** A message up to about 40 KB is sent immediately, without asking — a short request is done in one trip. A longer message is announced instead, and waits for the receiver.

4. **The receiver is the dispatcher.** For anything longer, the receiver sends **grants** ("send me the next 10 KB now"). The receiver is where traffic piles up when many machines answer at once (incast), and it is the only one that sees all of it, so it is the right one to decide. It favours the message that is **closest to finishing**, which minimises average waiting.
5. **The network's express lanes.** Datacenter switches already have up to 8 priority queues per port, mostly unused. Homa stamps each packet with a priority: short messages and control packets in the fastest lane, big transfers in the slow lanes. A small message overtakes the queue instead of waiting in it.

Lost packets are handled without connections: the receiver notices a missing piece and asks for it again (a **RESEND**).

The benefits

- **Much lower tail latency for short messages under load.** The Homa papers report 10–100× lower 99th-percentile latency than TCP for short messages in heavily loaded datacenter networks. This lab sees the same effect on a small scale: under incast, Homa's 100-byte messages stayed under 2 ms at the 99th percentile while TCP's took 20–40 ms (section 11).
- **No head-of-line blocking.** In a TCP stream, one lost or slow message holds up every message behind it on that connection. Homa's messages are independent.
- **Incast handled by design.** When 100 servers answer one client at the same moment, TCP's queues overflow; Homa's receiver lets them in a few at a time.
- **Scales to many peers.** No connection per pair of machines means no connection storms, no connection tables, no handshakes before the first byte.
- **It fits what applications actually do.** RPC frameworks build request/response on top of TCP's byte stream and have to re-find message boundaries; Homa gives them messages directly.
- **It uses hardware that is already there.** The priority queues exist in today's switches; Homa needs them configured, not replaced.

The drawbacks

- **Not in Linux yet.** It is an out-of-tree kernel module, tied to particular kernel versions (this lab pins 6.17.8 for that reason). Getting it into the mainline kernel has been under way since 2024, in stages, and the first upstream version will be deliberately incomplete.
- **Applications must change.** It is not a drop-in replacement: the API is message-based, receive buffers are memory the application maps, messages are limited to 1 MB. Libraries help (there is gRPC support from the same group), but nothing using TCP works over Homa unchanged.
- **Datacenter only.** It assumes a fast, low-latency, trusted network that one organisation controls, with congestion mostly at the receiver's link. It is not a protocol for the internet: there is no encryption, and NAT, firewalls, load balancers and cloud security groups that do not know IP protocol 146 will drop or mangle it.
- **It needs the switches configured.** Without priority queues, much of the advantage for short messages is gone — this lab measured it: one FIFO queue made Homa's short-message latency about 10× worse at the median and 4× at the 99th percentile (section 11).

- **Hardware offloads are built for TCP.** Network cards cut big TCP sends into packets in hardware; they don't do it for Homa. Homa works around this by disguising its packets as TCP ("TCP hijacking"), which some network security gear rejects — and which failed for large messages in this lab. Here TCP was faster for big transfers (section 10).
- **Big messages can be starved.** "Closest to finishing goes first" is great for small messages and hard on huge ones while small traffic keeps arriving. Homa has a knob that reserves bandwidth for the oldest message (`grant_fifo_fraction`); it is off by default in this build.
- **Tuned for very fast networks.** Its timers assume 25–100 Gbit/s. On this lab's 1 Gbit/s links, the defaults caused false "packet lost" alarms and failed requests under load until the timers were scaled (section 13).
- **A young ecosystem.** Fewer tools, fewer people who know it, fewer production deployments. Even the project's own Wireshark dissector was out of date (this lab ships its own).

So, when is it the right tool?

	TCP	Homa
unit of data	a byte stream	a message (≤ 1 MB)
setup	a connection per pair of machines	none
who controls the speed	each sender, by probing until loss	the receiver, with grants
switch priority queues	not used	essential
short messages under load	wait behind big transfers	overtake them
big transfers	hardware offload, efficient	no offload; can be starved
where it works	everywhere	inside one datacenter network
in Linux	since the beginning	out-of-tree module; upstreaming in progress

Homa makes sense for RPC-heavy services inside a datacenter whose owner controls the hosts and the switches, and where tail latency is the problem — storage systems, microservices, distributed databases, and machine-learning clusters with many small coordination messages. For anything crossing the internet, for bulk transfers, or where applications can't be changed, TCP (or QUIC) remains the answer.

3. Homa in five minutes

Messages, not streams

A Homa socket sends and receives whole **messages**: a request and its response form an **RPC**, identified by an id the client chooses (even numbers; the server answers with the id's low bit set). There are no connections — no handshake, no per-peer state to set up or tear down — and one socket can talk to any number of peers. Messages are limited to 1 MB (`HOMA_MAX_MESSAGE_LENGTH`). Applications use `sendmsg`/`recvmsg` on a `SOCK_DGRAM` socket with protocol 146; received messages land in a buffer region the application has mapped, so the kernel never copies them twice.

The receiver schedules

TCP's senders probe for bandwidth and back off when the network drops or marks packets; queues build up first. Homa inverts this. A message no longer than `unsched_bytes` (40,000 here, about one round trip's worth) is sent **unscheduled**, straight away. A longer one is **scheduled**: it waits for **GRANT** packets from the receiver, which knows every message heading its way and decides which to let in, and how fast. (Homa as published, and this module until September 2026, also sent the first `unsched_bytes` of a long message unscheduled; now a long message only announces itself with START_MSG and every byte is granted — section 8 shows it.) Because the receiver's downlink is where incast congestion happens, the receiver is in the best position to manage it.

SRPT, implemented with switch priorities

The receiver grants by **shortest remaining processing time**: the message closest to completion goes first. To make that real in the network, every Homa packet carries a priority (0–7, written into the top three bits of the IPv4 DSCP / IPv6 traffic class), and switches serve higher priorities first:

- **Unscheduled** packets (whole short messages) use the top priorities, chosen by message length through `unsched_cutoffs` (the shortest get the highest).
- **Scheduled** packets use the priority the GRANT names — the top scheduled priority (`max_sched_prio`, 6 here) for the message nearest completion, lower ones for the others.
- **Overcommitment**: the receiver grants several messages at once (up to `max_overcommit`) so its link never idles when one sender stalls; the priorities keep them from hurting each other.

A short message therefore jumps over the queued bulk data at the congested switch port. That is the property section 11 measures.

Reliability without connections

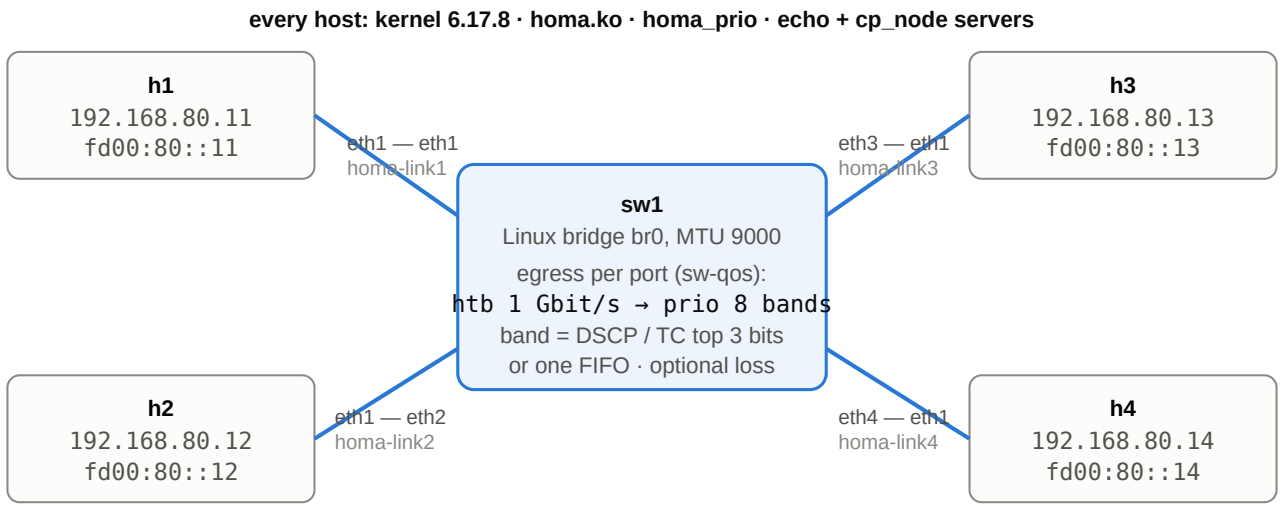
The receiver tracks which byte ranges of each message have arrived. If a granted range does not arrive within `resend_ticks` milliseconds, it sends a **RESEND**; a sender that has nothing to send yet answers **BUSY**. The client's request is acknowledged implicitly by the response; servers learn that their responses arrived through **ACK**s (piggybacked on later traffic, or prompted with **NEED_ACK**). An RPC with no sign of life for `timeout_ticks` fails with `ETIMEDOUT`.

The packet types

type	code	what it carries
`DATA`	0x10	a segment of a message: offset, the message length, a piggybacked ACK
`GRANT`	0x11	receiver → sender: "send up to this offset, at this priority"
`RESEND`	0x12	receiver → sender: "this range never arrived"
`RPC_UNKNOWN`	0x13	"I have no such RPC" (the peer restarted or the RPC is gone)
`BUSY`	0x14	"still here, nothing to send yet" (keeps an RPC from timing out)
`CUTOFFS`	0x15	the receiver's unscheduled priority cutoffs, so senders pick priorities it wants
`NEED_ACK` / `ACK`	0x17 / 0x18	the server asks whether its response arrived / the client confirms (up to 5 RPCs per packet)
`START_MSG`	0x19	announces a message that will be fully scheduled (new in September 2026: messages that need grants send no unscheduled data)

From `homa_wire.h` at the pinned commit. FREEZE (0x16) is a debugging aid. The lab's dissector decodes all of them.

4. Architecture



OOB management 10.8.0.0/24 on eth0 of every VM (h1–h4 .11–.14, sw1 .20; the host is .1)

Figure 1 — the topology. Each host↔sw1 link is its own isolated libvirt bridge; sw1 bridges the four into one L2 segment and applies its egress policy on the way out of each port, i.e. on the way *to* each host — the receiver's downlink, where incast congests.

Nodes

```
NODE ROLE    STATE      MGMT-IP     DATA-IP         DATA-IP6        CONSOLE  VM
h1   host    running    10.8.0.11   192.168.80.11   fd00:80::11     5701     homa-h1
h2   host    running    10.8.0.12   192.168.80.12   fd00:80::12     5702     homa-h2
h3   host    running    10.8.0.13   192.168.80.13   fd00:80::13     5703     homa-h3
h4   host    running    10.8.0.14   192.168.80.14   fd00:80::14     5704     homa-h4
sw1  switch  running    10.8.0.20   -               -               5705     homa-sw1

links (isolated libvirt bridges, MTU 9000):
  h1 eth1 192.168.80.11  <->  sw1 eth1   (homa-link1)
  h2 eth1 192.168.80.12  <->  sw1 eth2   (homa-link2)
  h3 eth1 192.168.80.13  <->  sw1 eth3   (homa-link3)
  h4 eth1 192.168.80.14  <->  sw1 eth4   (homa-link4)

image: homa-base.qcow2 — kernel 6.17.8-061708-generic, HomaModule 65cecd7; sw1 default policy:
prio 1gbit
```

Why each piece is the way it is

piece	choice	why
VMs, not containers	libvirt/KVM, 5 domains `homa-*`	a kernel module needs its own kernel; containers share the host's
kernel	mainline 6.17.8 (kernel.ubuntu.com) on Ubuntu 26.04	HomaModule's `main` is developed against 6.17.8; the host's 7.0 is "compiles, not actively supported"
module	HomaModule `main @ 65cecd7`	pinned in `lab.conf`; changing it is one line and `lab.sh image --force`
links	one isolated libvirt bridge per link, MTU 9000	the UDP tunnels the other labs use lost Homa's bursts in the host's socket buffer (section 18); bridges also give the virtio NICs TSO
bottleneck	sw1 shapes each port to 1 Gbit/s; hosts set `link_mbps=1000`	a defined, shared bottleneck at the receiver's downlink; Homa's default `link_mbps` assumes 25 Gbit/s
priority queues	`tc prio`, 8 strict bands, flower on `ip_tos`	what HomaModule's INSTALL.md asks of the switch: top 3 bits of DSCP / traffic class select the queue, priority 7 served first
sw1 offloads	GRO, GSO, TSO and `rx-gro-hw` off on its ports	a switch port queues wire-size frames, not 64 KB aggregates
servers	echo (4100/4101), cp_node (4000/5000), probe cp_node (4010/5010)	standing services; probes have their own processes so they never wait behind bulk messages in a server thread

5. How the image is built

`./lab.sh image` (`tools/build_image.sh`) turns the Ubuntu 26.04 cloud image into `images/homa-base.qcow2`; every node is a copy-on-write overlay on it. A throw-away **builder VM** on libvirt's NAT network (the lab networks have no internet) runs `image/homa-build.sh` once and powers off:

1. Disable unattended upgrades and hold the kernel packages — the lab must not change under its tests.
2. Install the four mainline 6.17.8 packages (image, modules, headers) and make GRUB boot it by name; the distribution kernel stays installed as a fallback.
3. Clone HomaModule, check out the pinned commit, `make` against the 6.17.8 headers, install `homa.ko` into `/lib/modules/6.17.8-061708-generic/updates/`, build `util/` (cp_node, homa_test, server, homa_prio, ...) into `/usr/local/bin`.
4. Build and run the module's **unit tests** — a user-space harness that compiles the module's sources against mocks of the kernel (840 tests). Two definitions are needed for the Ubuntu headers (section 18). The result is kept in `/var/log/homa-unit.result` and checked by suite 01.
5. Install the lab's pieces: `sw-qos`, the systemd units for homa_prio and the test servers, the Homa dissector for tshark. Generalise the disk (machine-id, SSH host keys, cloud-init state).

Each node then boots with its own cloud-init seed: addresses, the `lab` user with your SSH key, `/etc/hosts` (h1–h4, and `node1`–`node4`, the names cp_node expects), `homa` in `modules-load.d`, the Homa sysctls, and the services; sw1 gets its bridge and `sw-qos` policy instead.

```
+ make -j4 LINUX_VERSION=6.17.8-061708-generic KDIR=/lib/modules/6.17.8-061708-generic/build
'CINCLUDES=-I. -I.. $(KERN_INCLUDES) -include $(KDIR)/include/linux/kconfig.h -include lab-
kconfig.h'
++ grep -oE '[0-9]+ / [0-9]+ tests passed' /var/log/homa-unit.log
+ echo 'unit tests: PASS (840 / 840 tests passed)'
[==========] 840 / 840 tests passed.
+ echo 'dissector: homa.lua loaded'
dissector: homa.lua loaded
+ echo 'HOMA-BUILD-OK kernel=6.17.8-061708-generic homa=65cecd7831e8b24ba7bcd967288bb7ce26fe8eba
unit tests: PASS (840 / 840 tests passed)'
HOMA-BUILD-OK kernel=6.17.8-061708-generic homa=65cecd7831e8b24ba7bcd967288bb7ce26fe8eba unit
tests: PASS (840 / 840 tests passed)
HOMA-BUILD-RC=0
```

The end of the last image build (`images/build.log`).

```
[==========] 840 tests available from 18 test cases.
[==========] 840 / 840 tests passed.
[  PASSED  ]
unit tests: PASS (840 / 840 tests passed)
```

The unit-test result kept in the image.

```
filename:       /lib/modules/6.17.8-061708-generic/updates/homa.ko
alias:          net-pf-10-proto-146-type-2
alias:          net-pf-2-proto-146-type-2
version:        1.0
description:    Homa transport protocol
license:        Dual BSD/GPL
srcversion:     3642FFB922B111B1E58D279
name:           homa
vermagic:       6.17.8-061708-generic SMP preempt mod_unload modversions
```

`modinfo homa` on h1. The alias `net-pf-2-proto-146` is what makes the kernel load the module on the first Homa socket (suite 02 tests it).

6. Getting started

```
./lab.sh image        # once, ~10 min: builder VM → images/homa-base.qcow2
./lab.sh up           # define the networks and VMs, start them
./lab.sh wait         # until SSH answers and cloud-init is done (~1 min)
./lab.sh verify       # kernel + module, sw1 policy, jumbo pings, a Homa RTT between every pair
./lab.sh test         # all Robot suites (~4 min) → tests/results_<date>_<time>/report.html
./lab.sh down         # shut the VMs down (5 VMs, 9 GiB RAM, 10 vCPUs while running)
```

Log in with `./lab.sh ssh h1` (user `lab`, password `lab`, and your SSH key), or the serial console with `./lab.sh console h1`. `./lab.sh exec hosts '<command>'` runs a command on all four hosts in parallel. The Lab Hub (port 8088) shows the lab's VMs and can power it.

> **Everything is measured from the host.** `tools/homalab.py` is the single place that drives experiments: it reads `lab.sh inventory`, runs Homa's tools over SSH (every remote command carries its own `timeout`), reads sw1's queue counters and Homa's kernel metrics, and prints JSON. `lab.sh bench`, the Robot suites and this document all use it.

7. Walkthrough A — the lab comes up

After `up` and `wait`, `lab.sh homa status` confirms each host's kernel, the module and the priority daemon:

```
h1  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h2  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h3  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h4  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
```

`lab.sh verify` then checks the path end to end: sw1's policy and bridge ports, an unfragmentable 9000-byte ping both ways between every pair over IPv4 and IPv6, and a Homa round trip between every ordered pair — 12 pairs × IPv4/IPv6 × 100 B / 100 KB, one at a time (concurrent tests would measure the 2-vCPU hosts' overload, not Homa):

```
== hosts
h1  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h2  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h3  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000
h4  kernel 6.17.8-061708-generic  homa loaded (569344 B)  homa_prio active  link_mbps 1000

== sw1 egress policy
SW_QOS_MODE=prio
SW_QOS_RATE=1gbit
SW_QOS_LIMIT=200
br0 port eth1 forwarding
br0 port eth2 forwarding
br0 port eth3 forwarding
br0 port eth4 forwarding

== data plane: 8972-byte unfragmentable pings (MTU 9000), IPv4 and IPv6
  h1 -> h2  v4 ok  v6 ok
  h1 -> h3  v4 ok  v6 ok
  h1 -> h4  v4 ok  v6 ok
  h2 -> h1  v4 ok  v6 ok
  h2 -> h3  v4 ok  v6 ok
  h2 -> h4  v4 ok  v6 ok
  h3 -> h1  v4 ok  v6 ok
  h3 -> h2  v4 ok  v6 ok
  h3 -> h4  v4 ok  v6 ok
  h4 -> h1  v4 ok  v6 ok
  h4 -> h2  v4 ok  v6 ok
  h4 -> h3  v4 ok  v6 ok
  12/12 pairs ok

== Homa: round trips between every pair, IPv4 and IPv6, 100 B and 100 KB (homa_test -> the echo
servers)
  h1 -> h2 v4     100 B  p50    144.3 us  p90    206.5 us
  h1 -> h2 v4  100000 B  p50    810.0 us  p90    823.2 us
  h1 -> h3 v4     100 B  p50    136.1 us  p90    149.5 us
  h1 -> h3 v4  100000 B  p50    814.5 us  p90    835.0 us
  h1 -> h4 v4     100 B  p50    145.9 us  p90    187.0 us
  h1 -> h4 v4  100000 B  p50    810.5 us  p90    821.6 us
  h2 -> h1 v4     100 B  p50    133.2 us  p90    167.7 us
  h2 -> h1 v4  100000 B  p50    810.8 us  p90    820.1 us
  h2 -> h3 v4     100 B  p50    142.7 us  p90    163.3 us
  h2 -> h3 v4  100000 B  p50    810.3 us  p90    832.3 us
  h2 -> h4 v4     100 B  p50    135.5 us  p90    165.4 us
  h2 -> h4 v4  100000 B  p50    816.5 us  p90    857.5 us
  h3 -> h1 v4     100 B  p50    143.1 us  p90    154.5 us
  h3 -> h1 v4  100000 B  p50    815.2 us  p90    836.7 us
  h3 -> h2 v4     100 B  p50    129.8 us  p90    153.4 us
  h3 -> h2 v4  100000 B  p50    810.3 us  p90    831.3 us
  h3 -> h4 v4     100 B  p50    138.5 us  p90    168.9 us
  h3 -> h4 v4  100000 B  p50    811.5 us  p90    823.9 us
  h4 -> h1 v4     100 B  p50    140.1 us  p90    150.8 us
  h4 -> h1 v4  100000 B  p50    809.1 us  p90    843.8 us
```

```
h4 -> h2 v4     100 B  p50    130.3 us  p90    150.8 us
h4 -> h2 v4  100000 B  p50    816.7 us  p90    829.0 us
h4 -> h3 v4     100 B  p50    134.7 us  p90    163.5 us
h4 -> h3 v4  100000 B  p50    808.5 us  p90    822.9 us
48/48 Homa RTT tests ok
```

Output trimmed: only the IPv4 Homa lines are shown (the IPv6 ones look the same). A 100 B round trip takes about 0.15–0.25 ms between VMs; 100 KB each way about 0.8 ms at 1 Gbit/s.

8. Walkthrough B — one RPC on the wire

The lab captures with tcpdump on **sw1's port towards h1**, which sees both directions before either host touches them, and decodes with tshark and the lab's own dissector, `image/homa.lua` (HomaModule's `dissector/` still expects IP protocol 253 and the pre-2024 headers). Capturing on h1 itself would miss packets: Homa's GRO handler consumes many arriving GRANT and DATA packets before tcpdump's tap (section 18). `homa_test h2:4100 --count 1 --length 100 rtt` from h1 — it runs ten warm-up RPCs before the measured one:

```
0.000 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219372 off=0 len=100 msg=100
0.208 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219372 off=0 len=100 msg=100
0.305 ms   h1 → h2  prio 7     96 B  CUTOFFS 32842 → 4100 id=1219372
0.346 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219374 off=0 len=100 msg=100
0.439 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219374 off=0 len=100 msg=100
0.545 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219376 off=0 len=100 msg=100
0.670 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219376 off=0 len=100 msg=100
0.781 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219378 off=0 len=100 msg=100
0.865 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219378 off=0 len=100 msg=100
0.953 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219380 off=0 len=100 msg=100
1.042 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219380 off=0 len=100 msg=100
1.117 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219382 off=0 len=100 msg=100
1.201 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219382 off=0 len=100 msg=100
1.263 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219384 off=0 len=100 msg=100
1.342 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219384 off=0 len=100 msg=100
1.429 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219386 off=0 len=100 msg=100
1.505 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219386 off=0 len=100 msg=100
1.588 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219388 off=0 len=100 msg=100
1.690 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219388 off=0 len=100 msg=100
1.770 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219390 off=0 len=100 msg=100
1.859 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219390 off=0 len=100 msg=100
1.922 ms   h1 → h2  prio 7    190 B  DATA 32842 → 4100 id=1219392 off=0 len=100 msg=100
1.995 ms   h2 → h1  prio 7    190 B  DATA 4100 → 32842 id=1219392 off=0 len=100 msg=100
4.546 ms   h2 → h1  prio 7     62 B  NEED_ACK 4100 → 32842 id=1219392
4.660 ms   h1 → h2  prio 7    114 B  ACK 32842 → 4100 id=1219392 acks=1
```

Each RPC is exactly two packets: the request fits in one DATA packet at priority 7, and so does the response — no handshake, no connection, about 0.1–0.2 ms per round trip. The `id` is the client's RPC id (even, incremented by 2); h2's packets carry it with the low bit set, which the dissector folds back. A few milliseconds after the last response, h2 asks whether its responses arrived (NEED_ACK) and h1 confirms (ACK): only then may the server free its RPC state.

A 300 KB message

```
   0.000 ms   h1 → h2  prio 7     66 B  START_MSG 32843 → 4100 id=1219394 msg=300000
   0.187 ms   h2 → h1  prio 7     67 B  GRANT 4100 → 32843 id=1219394 granted=200000 prio=0
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=0 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=8924 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=17848 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=26772 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=35696 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=44620 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=53544 len=8924 msg=300000
   0.327 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=62468 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=71392 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=80316 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=89240 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=98164 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=107088 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=116012 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=124936 len=8924 msg=300000
   0.370 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=133860 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=142784 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=151708 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=160632 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=169556 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=178480 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=187404 len=8924 msg=300000
   0.475 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=196328 len=8924 msg=300000
   0.527 ms   h2 → h1  prio 7     67 B  GRANT 4100 → 32843 id=1219394 granted=300000 prio=0
   0.656 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=205252 len=8924 msg=300000
   0.656 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=214176 len=8924 msg=300000
   0.656 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=223100 len=8924 msg=300000
   0.656 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=232024 len=8924 msg=300000
   0.656 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=240948 len=8924 msg=300000
   0.657 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=249872 len=8924 msg=300000
   0.657 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=258796 len=8924 msg=300000
   0.657 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=267720 len=8924 msg=300000
   0.712 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=276644 len=8924 msg=300000
   0.712 ms   h1 → h2  prio 0   9014 B  DATA 32843 → 4100 id=1219394 off=285568 len=8924 msg=300000
   0.712 ms   h1 → h2  prio 0   5598 B  DATA 32843 → 4100 id=1219394 off=294492 len=5508 msg=300000
   2.141 ms   h2 → h1  prio 7     66 B  START_MSG 4100 → 32843 id=1219394 msg=300000
   2.238 ms   h1 → h2  prio 7     67 B  GRANT 32843 → 4100 id=1219394 granted=200000 prio=0
   2.357 ms   h2 → h1  prio 0   9014 B  DATA 4100 → 32843 id=1219394 off=0 len=8924 msg=300000
   2.368 ms   h2 → h1  prio 0   9014 B  DATA 4100 → 32843 id=1219394 off=8924 len=8924 msg=300000
   2.372 ms   h2 → h1  prio 0   9014 B  DATA 4100 → 32843 id=1219394 off=17848 len=8924 msg=300000
   2.375 ms   h2 → h1  prio 0   9014 B  DATA 4100 → 32843 id=1219394 off=26772 len=8924 msg=300000
   2.378 ms   h2 → h1  prio 0   9014 B  DATA 4100 → 32843 id=1219394 off=35696 len=8924 msg=300000
… 796 more packets
```

Now the receiver schedules. Since September 2026 a message that needs grants sends no unscheduled data at all: the sender announces it with `START_MSG` (just the length), and h2 answers one round trip later with a GRANT for the first 200,000 bytes. The DATA follows in bursts of full 9 KB frames, and h2 keeps extending the window (`granted=208924`, `226772`, …) as data drains. Every grant here says `prio=0`: with a single message incoming, Homa uses the *lowest* scheduled priority and keeps the higher ones free for shorter messages that may arrive. Packet counts for the run (11 RPCs, 300 KB each way): `{"DATA": 748, "GRANT": 68, "START_MSG": 22, "ACK": 1, "NEED_ACK": 1, "RESEND": 0, "BUSY": 0}`.

A GRANT, decoded

```
Homa GRANT
    Source port: 4100
    Destination port: 32843
    Sequence (TSO): 0
    Type: GRANT (0x11)
    Type name: GRANT
    Data offset (4-byte words, high nibble): 0x50
    Flags: 0x06
    Urgent (0xb97d: Homa's marker for TCP-hijacked packets; always set): 0xb97d
    Sender RPC id: 1219395
    [RPC id (client's): 1219394]
    [Sent by the client: False]
    Offset: 200000
    Priority: 0
```

A busier capture

`lab.sh bench capture` records sw1's port to h1 for four seconds while h2 streams 500 KB messages to h1 and h3 sends 100 B probes, then classifies every packet:

packet type	packets
DATA	109126
GRANT	43090
START_MSG	1204
NEED_ACK	726
ACK	724
RESEND	357
BUSY	254
RPC_UNKNOWN	21
CUTOFFS	7

what	priorities seen (0-7)
short messages (≤ 1 KB): their only packet	7
500 KB messages: the first packet	0, 1, 7
500 KB messages: data after 100 KB (scheduled)	0, 1, 7
priorities the GRANTs assign	0, 1
control packets (GRANT, RESEND, ACK, ...)	7

Short messages and every control packet travel at priority 7; the 500 KB messages' data travels at the low priorities the GRANTs assign (0 and 1 here — two messages compete, so the one nearer completion gets 1). The few long-message DATA packets at priority 7 are all **retransmissions**: in this capture, every one of them has the retransmit flag set — Homa resends at the priority the RESEND asks for. Suite 03 asserts these properties.

9. Walkthrough C — sw1's priority queues

Each of sw1's ports carries this tree, built by `sw-qos apply prio 1gbit 200` (Appendix B): an HTB class limits the port to 1 Gbit/s — the bottleneck — and feeds a `prio` qdisc with 8 bands, each a 200-packet FIFO. Flower filters pick the band from the top three bits of the IPv4 TOS / IPv6 traffic class: Homa priority 7 → band `10:1`, served first; priority 0 → `10:8`. Anything unclassified (ARP, neighbour discovery) goes to the last band.

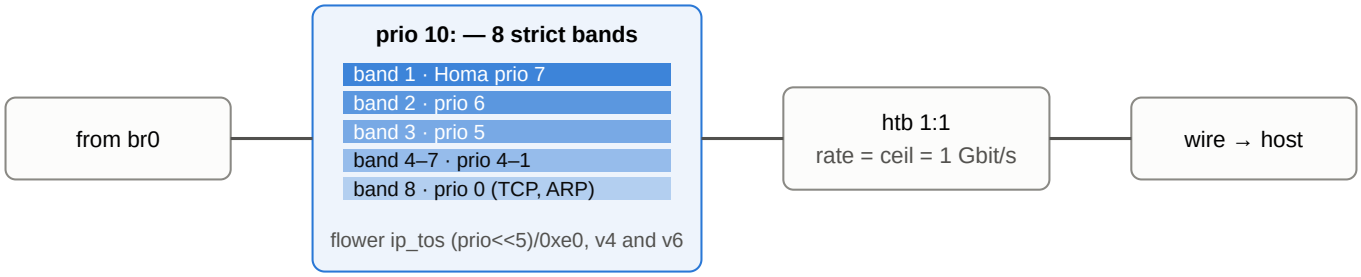


Figure 2 — one sw1 egress port in `prio` mode. In `fifo` mode the prio qdisc is replaced by one queue of 8 × 200 packets; in `off` mode there is no shaping at all.

After the busy capture in section 8, `lab.sh qos show eth1` shows the counters per band — the traffic sorted itself by the priority Homa wrote:

```
SW_QOS_MODE=prio
SW_QOS_RATE=1gbit
SW_QOS_LIMIT=200
== eth1
qdisc htb 1: root refcnt 2 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000
 Sent 620058967 bytes 90944 pkt (dropped 0, overlimits 67845 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 21: parent 10:1 limit 200p
 Sent 11882183 bytes 22312 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 23: parent 10:3 limit 200p
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 25: parent 10:5 limit 200p
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc prio 10: parent 1:1 bands 8 priomap 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
 Sent 620058967 bytes 90944 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 27: parent 10:7 limit 200p
 Sent 434595378 bytes 49275 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 22: parent 10:2 limit 200p
 Sent 180 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 26: parent 10:6 limit 200p
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 24: parent 10:4 limit 200p
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 28: parent 10:8 limit 200p
 Sent 173581226 bytes 19355 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc clsact ffff: parent ffff:fff1
 Sent 19253577 bytes 2916 pkt (dropped 56, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
```

`lab.sh qos stats` gives the same as JSON, per port and per Homa priority; the experiments read it before and after each run.

10. Walkthrough D — Homa against TCP

`lab.sh bench compare` runs HomaModule's `cp_node` client on h1 against the cp_node servers on h2–h4, first over Homa, then over TCP, with the same workload: message sizes drawn from Homa's **W3** distribution (mostly small messages, a long tail of large ones), 0.3 Gbit/s offered, up to 8 RPCs outstanding, 12 s each. cp_node records every RPC's round-trip time; the lab groups them by size.

messages	Homa (ms)				TCP (ms)			
	n	p50	p99	p99.9	n	p50	p99	p99.9
short (≤1 KB)	145668	0.29	2.32	7.16	154322	0.32	4.00	12.0
medium (1-100 KB)	23420	0.33	2.61	7.65	24849	0.34	4.09	13.2
long (>100 KB)	801	3.09	20.9	26.0	799	2.14	15.9	18.0

Round-trip times in ms. n = RPCs in the class.

At this light load the medians are close — within about 10%, one way or the other between runs — but Homa's tail for short and medium messages is lower: its p99 is roughly 0.6× TCP's. TCP is better for long messages here, and the reason is specific to this setup: the virtio NICs give TCP segmentation offload (TSO), so TCP hands the NIC 64 KB at a time, while Homa sends 9 KB packets one by one (its own software GSO is "temporarily broken" per its INSTALL.md, and its TCP-hijacking mode failed for large messages here — section 17). Suite 04 records this run and only fails if Homa's short-message p99 is more than 1.5× TCP's.

11. Walkthrough E — incast

This is the experiment the lab was built around. **h2 and h3** stream 500 KB messages to **h1** as fast as they can (one-way, four outstanding each) — together about twice what h1's 1 Gbit/s port at sw1 can take, so a queue builds there. Meanwhile **h4** sends 100-byte probes to h1, one at a time, to a separate server process, and records each probe's round trip. The run is repeated with sw1's strict priority queues and with one FIFO, for Homa and for TCP (`lab.sh bench incast`).

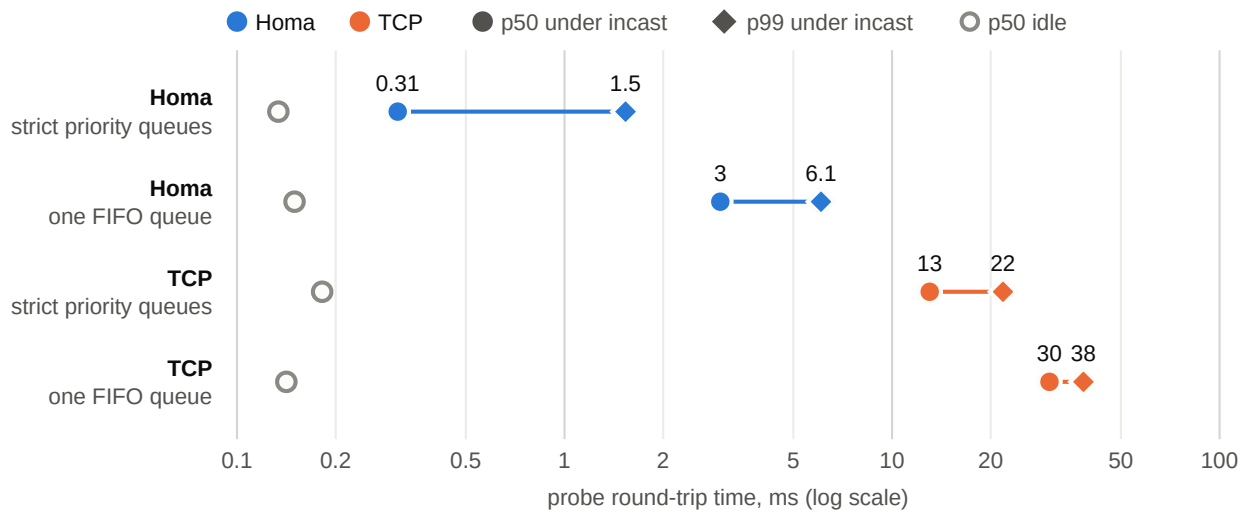


Figure 3 — probe round-trip times under incast. Hollow gray: the probe's median with no load. Filled dot: median under incast; diamond: 99th percentile. Log scale.

sw1 queues	protocol	probe p50 idle (ms)	probe p50 under incast	probe p99 under incast	probes	bulk Mbit/s	sw1 drops
strict priority	Homa	0.13	0.31	1.53	25957	967	8114
strict priority	TCP	0.18	13.0	21.8	770	989	58
one FIFO	Homa	0.15	2.99	6.07	3272	980	0
one FIFO	TCP	0.14	30.3	38.4	331	987	0

- **Homa with priority queues**: the probes travel at priority 7 and overtake the queued bulk data at sw1; their median stays within a few hundred microseconds of idle.
- **Homa with one FIFO**: the probes wait behind whatever Homa's grants have let into the queue — milliseconds. Homa's receiver-side scheduling still keeps that queue bounded.
- **TCP**: everything is priority 0, so the priority queues change nothing; the probes wait behind the TCP flows' standing queue (tens of milliseconds) in both modes.

Suite 05 asserts the bulk senders saturate the port in every run, that priority queues lower Homa's probe p99 and cut its median at least threefold, and that Homa's probes beat TCP's by 10× at the median and 3× at p99. Homa's p99 with priority queues varies between runs (1.5–4.5 ms observed) — the reason is section 13. The one cost visible here is that sw1 drops packets in priority mode and not in FIFO mode — also section 13.

12. Walkthrough F — loss recovery

`sw-qos loss eth1 50` makes sw1 drop one in 50 packets leaving towards h1 — any packet: data, grants, acknowledgements (a `clsact` egress filter with a random `gact drop`). h2 then runs 300 round trips of 20 KB each way with h1 (`lab.sh bench loss`), and the lab reads Homa's counters before and after:

	value
round trips (20 KB each way), completed	300
RTT p50 / p90 (ms)	0.30 / 0.36
RESENDs sent by h1 (receiver of the requests)	19
packets retransmitted by h2	20
RESENDs sent by h2 (receiver of the replies)	19
RPC timeouts (h1 / h2)	0 / 0

Every RPC completed with the right contents. Homa has no connection state to repair: h1, receiving h2's requests, noticed gaps in the byte ranges and sent RESENDs; h2 retransmitted exactly those ranges. Losses towards h1 also hit the GRANTs and ACKs h1 receives, which the protocol absorbs the same way. Suite 06 asserts all RPCs complete, RESENDs were sent and answered, and nothing timed out.

13. Walkthrough G — the resend timers

The incast runs in priority mode showed sw1 dropping thousands of packets that FIFO mode never dropped, and Homa logging RPC timeouts, with no loss injected. The per-band counters located it: the drops were in the **lowest** bands. The mechanism:

1. Two senders push 500 KB messages at h1. SRPT grants the message nearest completion at the top scheduled priority and **parks the others** at lower priorities (overcommitment).
2. Strict priority serves the top band first. At 1 Gbit/s a 500 KB message takes 4 ms to drain, and the next one starts behind it — so the parked messages' packets sit in the low bands for **tens of milliseconds**.
3. Homa's receiver hears nothing from them for `resend_ticks` = 5 ms and sends RESEND. The data is not lost — only queued — but the sender sends it again, into the same starved band.
4. The band overflows (sw1 drops), real gaps appear, more RESENDs follow; some RPCs exhaust `timeout_ticks` and fail.

Homa's defaults assume a 25–100 Gbit/s network, where the same bytes drain 25–100 times faster and a 5 ms silence really means loss. Scaling the timers by ten for this 1 Gbit/s network removes the effect (`lab.sh homa sysctl resend_ticks=50 resend_interval=20 timeout_ticks=1000`):

timers	bulk Mbit/s	sw1 drops	RESENDs from h1	RPC timeouts
Homa defaults: resend_ticks 5, resend_interval 5, timeout_ticks 100	875	5814	2164	22
×10: resend_ticks 50, resend_interval 20, timeout_ticks 1000	877	0	302	0

8 s of bulk from h2 and h3 into h1, sw1 in priority mode, 200 packets per band.

The lab keeps Homa's defaults — they are what is under test — and suite 07 holds the finding: with the defaults there are spurious RESENDs; with the scaled timers there are no drops, no timeouts and at most a third of the RESENDs. A larger queue does not help (tried: 400, 800, 1600 packets per band — still drops and timeouts): the retransmissions grow to fill it.

14. Walkthrough H — segmentation offload

A sender saves CPU by handing its NIC one large frame and letting the NIC (or the kernel, just before the NIC) cut it into packets: **segmentation offload**. TCP gets it from virtio as TSO. Homa has two ways to ask for it, and with either, every message bigger than one packet was lost here. The two causes turned out to be different.

configuration	message	result	p50 (ms)	frames sent	packets received
defaults (one frame per packet)	1 KB	ok	0.14	15	15
defaults (one frame per packet)	10 KB	ok	0.18	30	30
defaults (one frame per packet)	1 MB	ok	15.4	1695	1695
native GSO, guest TSO on	1 KB	ok	0.14	15	15
native GSO, guest TSO on	10 KB	**lost**	–	20	0
native GSO, guest TSO on	1 MB	**lost**	–	156	0
native GSO, guest segments (TSO off)	1 KB	ok	0.16	15	15
native GSO, guest segments (TSO off)	10 KB	ok	0.22	15	30
native GSO, guest segments (TSO off)	1 MB	ok	15.5	255	1695
TCP hijacking	1 KB	ok	0.16	15	15
TCP hijacking	10 KB	**lost**	–	11938	569
TCP hijacking	1 MB	**lost**	–	897	0

Round trips h2 → h1, 5 per size. "frames" is what h2 handed its NIC (Homa's `packets_sent_DATA`), "packets" what h1 received: fewer frames than packets means offload worked. From `./lab.sh bench offload`; suite 08 checks the same.

Native GSO: the lab's wiring

1. With `max_gso_size=100000` Homa builds GSO frames of up to 64 KB. The guest's virtio NIC advertises TSO, so the guest kernel hands the frame over whole.
2. The frame reaches the lab host's tap. The host has to cut it into packets for sw1, but it has no Homa module and can't segment IP protocol 146. It drops the frame: the tap's `tx_dropped` grew by exactly the number of GSO frames.
3. With TSO off on the guest NIC (configuration `gso_guest`), the guest segments the frame itself with Homa's `homa_gso_segment`, and every size works. That's despite INSTALL.md calling software GSO "temporarily broken".

TCP hijacking: a bug in the module

1. With `hijack_tcp=1` Homa disguises its packets as TCP so the NIC's TSO cuts them. The receiver's GRO hook must recognise the disguise and turn the packets back into Homa.

2. The check reads the header through `skb_transport_header()`. For a full-size packet virtio copies only the Ethernet header into the skb's linear area; the rest stays in a page fragment. The check reads the wrong bytes and says "not Homa".
3. TCP receives a packet whose flags include RST and drops it: a kernel drop trace counted 1,701 `TCP_RESET` drops for one test. Short packets are copied whole, so control packets and one-packet messages work.
4. `patches/0001-hijack-gro-read-header-from-frag0.patch` reads the header the GRO way (`skb_gro_header()`) before deciding. With it every size from 1 KB to 1 MB works, and 1 MB goes out as 510 TSO frames. The lab still runs the unpatched pinned module; `patches/README.md` has the bug report for upstream.

> **Two traps on the way.** `hijack_tcp` is read when a socket is created, so servers started before the change don't hijack: an earlier "hijacking works for small RPCs" result came from a hijacking client and a plain server. And `ethtool -K tso` is a feature group: switching it on also switches on `tx-tcp-mangleid-segmentation`, so restoring the defaults takes two calls.

What it changed in the findings. With offload working for Homa (patched module, hijacking with GSO), Homa's long-message p50 on the W3 workload stayed at about 3.2 ms against TCP's 2.1–2.4 ms. So TSO does not explain TCP's lead on long messages (section 10); that remains open.

15. Configuration and metrics

Homa's sysctls

Homa's configuration lives under `net.homa` (documented in `man 7 homa`, installed on the hosts). The lab sets only `link_mbps=1000` (`/etc/sysctl.d/90-homa.conf`, from `HOMA_SYSCTL` in lab.conf); `unsched_cutoffs` and `cutoff_version` are maintained by homa_prio. `lab.sh homa sysctl name=value` sets one on all hosts.

```
net.homa.accept_bits = 0
net.homa.action = 0
net.homa.bpage_lease_usecs = 10000
net.homa.busy_usecs = 100
net.homa.cutoff_version = 13
net.homa.dead_frags_limit = 100
net.homa.defer_min_bytes = 1000
net.homa.drop_bits = 0
net.homa.fifo_grant_increment = 50000
net.homa.freeze_type = 0
net.homa.gen3_softirq_cores = 0 1       -1      -1
1       0       -1      -1
net.homa.grant_fifo_fraction = 0
net.homa.gro_busy_usecs = 5
net.homa.gro_policy = 114
net.homa.gso_force_software = 0
net.homa.hijack_tcp = 0
net.homa.homa_share = 50
net.homa.link_mbps = 1000
net.homa.max_dead_frags = 35
net.homa.max_grantable_rpcs = 8
net.homa.max_gro_skbs = 20
net.homa.max_gso_size = 1000
net.homa.max_incoming = 400000
net.homa.max_link_usage = 99
net.homa.max_nic_est_backlog_usecs = 5
net.homa.max_nic_queue_usecs = 40
net.homa.max_overcommit = 8
net.homa.max_sched_prio = 6
net.homa.next_id = 0
net.homa.num_priorities = 8
net.homa.pacer_fifo_fraction = 50
net.homa.peer_gc_threshold = 5000
net.homa.peer_idle_secs_max = 120
net.homa.peer_idle_secs_min = 10
net.homa.peer_net_max = 10000
net.homa.poll_usecs = 50
net.homa.priority_map = 0       1       2       3
4       5       6       7
net.homa.request_ack_ticks = 2
net.homa.resend_interval = 5
net.homa.resend_ticks = 5
net.homa.temp = 0       0       0       0
net.homa.timeout_resends = 5
net.homa.timeout_ticks = 100
net.homa.tx_page_frees_per_sec = 1
net.homa.tx_page_pool_min_kb = 3000
net.homa.unsched_bytes = 40000
net.homa.unsched_cutoffs = 2147483647   1000000 1000000
1000000 1000000 1000000 1000000 1000000
net.homa.verbose = 0
net.homa.window = 0
net.homa.wmem_max = 100000000
```

Homa's metrics

`/proc/net/homa_metrics` holds about 700 counters per core: packets sent and received by type, bytes and packets per priority, retransmissions, timeouts, pacer and lock statistics, message-size histograms. `lab.sh bench metrics h1` sums them over cores; the experiments diff them. The ones this lab relies on:

counter	meaning	used by
`packets_sent_* / packets_rcvd_*`	per packet type (DATA, GRANT, RESEND, BUSY, ...)	loss, timers
`priorityN_packets / _bytes`	packets sent at each priority	cross-check with sw1's bands
`resent_packets`	DATA packets retransmitted after a RESEND	loss
`rpc_timeouts`	RPCs abandoned after `timeout_ticks`	loss, timers
`client_requests_done, server_responses_done`	completed messages	sanity

16. Tests

`./lab.sh test` runs Robot Framework over `tests/suites`; the keywords (`tests/resources/HomaLab.py`) wrap `tools/homalab.py`. A run takes about 5 minutes and leaves `tests/results_<date>_<time>/` with Robot's report and log and `state/` — every host's Homa sysctls and module version and sw1's policy before the run. Runs take a lock (`/tmp/homa-lab-test.lock`); suites that change sw1 or sysctls put them back in their teardown.

suite	what it checks
01 platform	the pinned kernel; the loaded module is the installed build at the pinned commit; unit tests passed in the image; sysctls, metrics, protocol-146 sockets (v4/v6); services; sw1's bridge and policy
02 function	jumbo frames between every pair; 48 Homa round trips (12 ordered pairs × v4/v6 × 100 B/100 KB); sizes 8 B–1 MB intact; unload refused with sockets open; unload, and autoload on the first Homa socket
03 wire	decoded capture: DATA/GRANT/START_MSG/ACK present; short messages and control at priority 7; scheduled data at lower, granted priorities; sw1 used several queues
04 compare	Homa vs TCP on W3; both complete; Homa's short-message p99 not worse than 1.5× TCP's (warns if above TCP's)
05 incast	bulk saturates h1's port; priority queues help Homa's probes; Homa beats TCP 10× at p50 and 3× at p99; TCP stays in the lowest queue
06 loss	1/50 loss: every RPC completes; RESENDs and retransmissions; no timeouts
07 timers	default timers cause spurious RESENDs; ×10 timers: no drops, no timeouts, ≥3× fewer RESENDs
08 offload	defaults: one frame per packet; native GSO works when the guest segments and is lost on the host with TSO on; the hijack bug (tagged known-bug: fails once the pinned module is fixed)

The last run

suite	passed	time
01 Platform	5/5	13 s
02 Function	5/5	26 s
03 Wire	6/6	17 s
04 Compare	2/2	29 s
05 Incast	4/4	78 s
06 Loss	2/2	2 s
07 Timers	2/2	21 s
08 Offload	4/4	80 s

Run results_2026-10-15_21-29-45: 30 passed, 0 failed.

17. Findings

1. **Priority queues do what Homa's design says.** With h1's port saturated, Homa's short messages keep near-idle latency when the switch has strict priority queues, and lose an order of magnitude without them. TCP cannot use the queues at all (section 11).
2. **Homa's timers are tuned for 25–100 Gbit/s.** At 1 Gbit/s, SRPT plus strict priority starves parked messages longer than `resend_ticks`; the spurious retransmissions overflow the switch and RPCs time out. Timers scaled ×10 remove it (section 13). Anyone running Homa on slower links (or emulating it) should scale `resend_ticks`, `resend_interval` and `timeout_ticks` with the link speed.
3. **Light load:** Homa's short-message tail is about 0.6× TCP's, medians are similar, and TCP wins long messages (section 10). Not because of TSO: with offload working for Homa, its long-message p50 didn't move (section 14). The cause is open.
4. **Offloads (section 14):** native GSO is lost on the lab host when the guest NIC does TSO (the host can't segment Homa), and works when the guest segments; TCP hijacking loses every full-size packet because of a receive-side bug in the module (a patch is in `patches/`). The lab keeps the defaults.
5. **The unit tests** do not compile against Ubuntu's mainline headers as shipped; with two definitions they all pass (section 18).
6. **Module lifecycle is clean:** it refuses to unload while sockets are open, unloads without residue, and is loaded again automatically by the first protocol-146 socket.
7. **A malformed request lingers by design.** A message shorter than the echo protocol's 8-byte header leaves the server waiting for a reply length it never gets; the client RESENDs, the server answers BUSY, indefinitely, until the client exits. Correct protocol behaviour — the server is "busy" — but a reason every lab command carries a timeout.

18. Lessons learned building it

The host was dropping the lab's packets

The first version linked the VMs with QEMU UDP-socket tunnels, as the other labs do. Homa ran, but with thousands of RESENDs and dozens of timeouts and no loss anywhere in the lab — sw1's counters showed no drops, the hosts' interfaces showed no errors. The host's `UdpRcvbufErrors` counter rose by 1,641 during one 100-RPC run: QEMU's socket backend leaves the host's default 208 KB receive buffer, about 23 jumbo frames, and Homa's bursts overflowed it. Raising `net.core.rmem_default` would have needed root and changed every lab on the host; isolated libvirt bridges fixed it locally and, as a bonus, gave the virtio NICs TSO. 1 MB each way went from 25.8 ms to 15.4 ms (line rate).

The unit tests and the distribution headers

HomaModule's unit-test harness compiles the module's sources in user space against the kernel headers with mocks. Against the Ubuntu mainline 6.17.8 headers it fails twice: the headers enable the `__seg_gs` per-CPU address space, which `mock.h`'s plain `current_task` declaration conflicts with; and they turn

`preempt_count_add/sub` into macros, which `mock.c` defines as functions. The build adds a header (`test/lab-kconfig.h`) included after `kconfig.h`:

```
#undef CONFIG_USE_X86_SEG_SUPPORT
#undef CONFIG_CC_HAS_NAMED_AS
#define CONFIG_TRACE_PREEMPT_TOGGLE 1
```

— presumably matching the author's own kernel configuration. With it, 840 of 840 pass.

Measure the network, not the application

The first incast runs showed TCP probes faster than Homa's. Two artefacts: the probes went to the same single-threaded cp_node server as the bulk messages, so they waited in the application; and two senders at ~0.4 Gbit/s each (on the old links) never filled a 1 Gbit/s port, so no queue formed at sw1 at all — the priority queues had nothing to do. Separate probe servers and a port that is actually oversubscribed fixed both. Likewise, running the 48 round-trip tests concurrently produced 27 ms latencies and timeouts — the 2-vCPU hosts were overloaded; one at a time they take 0.2 ms.

A switch port sees wire-size frames

With TSO on the hosts, sw1's virtio NICs received TCP's 64 KB segments as single "packets" (`rx-gro-hw`), so a 200-packet queue held 12 MB of TCP and the counters were meaningless. sw-qos turns off GRO, GSO, TSO and `rx-gro-hw` on sw1's ports so the host segments before delivering.

Capture where nothing has consumed the packets yet

The first traces, taken on h1, showed requests leaving but no responses or GRANTs arriving — yet the RPCs completed. Homa's GRO handler processes many arriving packets itself, ahead of the normal receive path where tcpdump taps: on h1, `gro_grant_bypasses` was 24,666 of 35,304 received GRANTs, and `gro_data_bypasses` about a fifth of the DATA. The lab captures on sw1 instead. (The dissector also had to use the packet's reported length, not the captured one, to show segment sizes from a 128-byte snaplen.)

tshark and AppArmor

Ubuntu's AppArmor profile for tshark does not let it read files in `/tmp` or a home directory, nor load plugins from `~/.local` — it fails with "permission denied" on a file it plainly can read. The lab pipes captures in (`sudo cat x.pcap | tshark -r -`) and installs the dissector in the global plugin directory.

19. Troubleshooting

symptom	cause and fix
`lab.sh wait` says NOT ready	still booting: SSH accepts the key but `pam_nologin` refuses logins until boot ends. `wait` retries for 6 minutes; check `./lab.sh log h1`.
`homa_test`: `Connection timed out`	the peer's server is not running (`systemctl status homa-echo`), the module was unloaded, or the message is under 8 bytes (the echo protocol's header).
a stream of RESEND/BUSY with no test running	an orphaned client: `./lab.sh exec hosts 'pkill homa_test; pkill -x cp_node'` (the standing servers are systemd units and restart).
`modprobe -r homa`: in use	sockets are open — stop the units first: `lab.sh homa unload` does it.
sw1 counters look wrong after a test	a suite was interrupted before its teardown: `lab.sh qos apply prio 1gbit 200`, `lab.sh qos loss eth1 off`, and the Homa timers: `lab.sh homa sysctl resend_ticks=5 resend_interval=5 timeout_ticks=100`.
tshark: permission denied	AppArmor (section 18): `sudo cat file | tshark -r -`.
Robot run waits at the start	another run holds `/tmp/homa-lab-test.lock`.
changing the module or kernel	edit `HOMA_COMMIT` / `KERNEL_*` in lab.conf, `lab.sh down; lab.sh clean; lab.sh image --force; lab.sh up`. Never rebuild the image under running overlays.

20. What is not built yet

- A second kernel track built from the upstream Homa patch series, to compare the stripped upstream version with the full module.
- Reporting the hijack bug upstream (draft in `patches/README.md`), and finding why Homa's long messages are slower than TCP's here.
- HomaModule's cluster benchmarks (`cp_vs_tcp`, `cp_load`) adapted to four nodes.

Appendix A — reference: lab.sh

```
  image [--force]      build images/homa-base.qcow2 (Ubuntu 26.04 + kernel 6.17.8-061708-generic
+ homa.ko 65cecd7), ~10 min
  up [node..]          create (if needed) and start the VMs (default: all)
  wait [node..]        until SSH answers and cloud-init is done
  down [node..]        shut the VMs down
  status | inventory   nodes, addresses, links | the lab as JSON
  homa status|load|unload|reload|sysctl [k[=v]..]|metrics [host]   the module on the hosts
  qos <args>           sw1's egress policy: show | apply prio|fifo|off [RATE] [LIMIT] [PORT..] |
loss PORT N|off | stats
  exec <node|hosts|all> <cmd>   run a command (hosts/all: in parallel)
  bench <sub> [opts]   rtt SRC DST | compare | incast | loss | capture | metrics HOST
(tools/homalab.py --help)
  verify               module + kernel, sw1 policy, jumbo ping matrix v4/v6, one Homa RPC per
host pair
  test [robot args]    Robot Framework suites, results/<timestamp>/
  ssh <node> [cmd] | console <node> | log <node> [n]
  rebuild [node..]     re-define domains and cloud-init seeds (keeps disks; stop the nodes first)
  clean [node..]       destroy VMs and delete their disks (base image kept)
nodes: h1 h2 h3 h4 sw1   (libvirt domains homa-<node>; user lab / lab, your SSH key installed)
```

tools/homalab.py

```
"""Drive the Homa lab from the host: the inventory, SSH to the VMs, Homa round trips, cp_node
workloads, sw1's queues,
packet captures and Homa's kernel metrics. Used by lab.sh (verify, bench) and by the Robot suites
(tests/resources).

  homalab.py matrix [--count N]                  one Homa RTT test per host pair, IPv4 and IPv6
  homalab.py rtt SRC DST [--length B] [--count N] [--ipv6]
  homalab.py compare [--workload w3] [--gbps 0.3] [--seconds 12]      Homa vs TCP, same workload,
h1 -> h2..h4
  homalab.py incast [--modes prio,fifo] [--protocols homa,tcp] [--seconds 10]   bulk h2,h3 -> h1
+ probes h4 -> h1
  homalab.py loss [--n 50] [--count 300]        drop 1/N on sw1 towards h1; Homa recovers
(RESENDs, no failures)
  homalab.py capture [--seconds 4]              one mixed run into h1, captured on sw1, decoded
(types, priorities)
  homalab.py offload [--configs a,b] [--sizes 1000,..]   round trips per segmentation-offload
setting (GSO, hijacking)
  homalab.py metrics HOST                       /proc/net/homa_metrics summed over cores (JSON)
```

Appendix B — sw-qos

```
#!/bin/bash
# sw1's egress policy on its host ports: what a datacenter top-of-rack switch does to Homa's packets.
#   sw-qos apply prio|fifo|off [RATE] [LIMIT] [PORT..]   prio: htb RATE -> prio, 8 strict bands chosen by the
top 3 bits of
#                                                        DSCP / IPv6 traffic class (Homa priority 7 = first
band), LIMIT
#                                                        packets each; fifo: htb RATE -> one queue of 8 x LIMIT;
off: no shaping
#   sw-qos loss PORT N|off        drop 1 out of N packets leaving PORT (clsact egress, gact netrand), any
protocol
#   sw-qos show [PORT..]          the qdiscs, per-band counters (sent / dropped / backlog) and the loss filter
#   sw-qos stats [PORT..]         one JSON object: per port, per Homa priority, packets / bytes / drops
set -euo pipefail
PORTS_ALL=(eth1 eth2 eth3 eth4)
die() { echo "sw-qos: $*" >&2; exit 1; }
ports() { [[ $# -gt 0 ]] && echo "$*" || echo "${PORTS_ALL[*]}"; }

apply_port() {   # port mode rate limit
  local p="$1" mode="$2" rate="$3" limit="$4" prio band
  # queue wire-size frames, as a switch port sees them: no GRO, and no 64 KB virtio receive aggregates (rx-gro-
hw)
  ethtool -K "$p" gro off gso off tso off rx-gro-hw off >/dev/null 2>&1 || true
  tc qdisc del dev "$p" root 2>/dev/null || true
  case "$mode" in
    off) ;;
    fifo)
      tc qdisc add dev "$p" root handle 1: htb default 1
      tc class add dev "$p" parent 1: classid 1:1 htb rate "$rate" ceil "$rate" quantum 9014 burst 90k cburst
90k
      tc qdisc add dev "$p" parent 1:1 handle 10: pfifo limit $(( limit * 8 )) ;;
    prio)
      tc qdisc add dev "$p" root handle 1: htb default 1
      tc class add dev "$p" parent 1: classid 1:1 htb rate "$rate" ceil "$rate" quantum 9014 burst 90k cburst
90k
      # strict priority: band 10:1 is served first. Unclassified traffic (ARP, ND) -> the lowest band via the
priomap.
      tc qdisc add dev "$p" parent 1:1 handle 10: prio bands 8 priomap 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
      for prio in 7 6 5 4 3 2 1 0; do
        band=$(( 8 - prio ))                                  # Homa priority 7 -> 10:1 ... priority 0 ->
10:8
        tc filter add dev "$p" parent 10: protocol ip   prio 1 flower ip_tos "$(( prio << 5 ))/0xe0" classid
"10:$band"
        tc filter add dev "$p" parent 10: protocol ipv6 prio 2 flower ip_tos "$(( prio << 5 ))/0xe0" classid
"10:$band"
        tc qdisc add dev "$p" parent "10:$band" handle "$(( 20 + band )):" pfifo limit "$limit"
      done ;;
    *) die "mode must be prio, fifo or off" ;;
  esac
}

cmd_apply() {
  local mode="${1:?mode}" rate="${2:-1gbit}" limit="${3:-200}"; shift 3 2>/dev/null || shift $#
  local p; for p in $(ports "$@"); do apply_port "$p" "$mode" "$rate" "$limit"; done
  printf 'SW_QOS_MODE=%s\nSW_QOS_RATE=%s\nSW_QOS_LIMIT=%s\n' "$mode" "$rate" "$limit" > /run/sw-qos.state
  echo "sw-qos: $mode rate=$rate limit=$limit on $(ports "$@")"
}

cmd_loss() {
```

```
  local p="${1:?port}" n="${2:?N or off}"
  tc qdisc add dev "$p" clsact 2>/dev/null || true
  tc filter del dev "$p" egress 2>/dev/null || true
  [[ "$n" == off ]] && { echo "sw-qos: no loss on $p"; return; }
  [[ "$n" =~ ^[0-9]+$ && "$n" -ge 2 ]] || die "N must be an integer >= 2 (drop 1 out of N)"
  tc filter add dev "$p" egress matchall action gact pass random netrand drop "$n"
  echo "sw-qos: dropping 1/$n of the packets leaving $p"
}

cmd_show() {
  local p; [[ -f /run/sw-qos.state ]] && cat /run/sw-qos.state
  for p in $(ports "$@"); do echo "== $p"; tc -s qdisc show dev "$p"; tc -s filter show dev "$p" egress
2>/dev/null | grep -E 'gact|random|Sent' || true; done
}

cmd_stats() {   # parse `tc -s -j qdisc` into {port: {mode, prio: {7: {...}, ...}, total: {...}}}
  python3 - "$(ports "$@")" <<'PY'
import json, subprocess, sys
out = {}
for port in sys.argv[1].split():
    q = json.loads(subprocess.run(["tc", "-s", "-j", "qdisc", "show", "dev", port], capture_output=True,
text=True).stdout or "[]")
    bands, total = {}, None
    for e in q:
        h, parent = e.get("handle", ""), e.get("parent", "")
        st = {"packets": e.get("packets", 0), "bytes": e.get("bytes", 0), "drops": e.get("drops", 0), "backlog":
e.get("backlog", 0), "qlen": e.get("qlen", 0)}
        if e.get("kind") == "pfifo" and parent.startswith("10:"):
            band = int(parent.split(":")[1], 16)
            bands[str(8 - band)] = st
        if e.get("kind") in ("htb", "pfifo_fast", "fq_codel", "noqueue", "mq") and h == "1:" or (e.get("root")
and total is None):
            total = st
    mode = "prio" if bands else ("fifo" if any(e.get("kind") == "pfifo" for e in q) else "off")
    out[port] = {"mode": mode, "prio": bands, "total": total}
print(json.dumps(out))
PY
}

sub="${1:-show}"; shift || true
case "$sub" in
  apply) cmd_apply "$@" ;;
  loss)  cmd_loss "$@" ;;
  show)  cmd_show "$@" ;;
  stats) cmd_stats "$@" ;;
  *) die "usage: sw-qos apply prio|fifo|off [RATE] [LIMIT] [PORT..] | loss PORT N|off | show [PORT..] | stats
[PORT..]" ;;
esac
```

Appendix C — lab.conf and layout

```
# Homa lab: four Linux hosts running the Homa transport protocol as an out-of-tree kernel module
(PlatformLab/HomaModule),
# joined by a Linux switch VM (sw1) whose egress ports model a datacenter switch: a rate limit (the bottleneck)
and 8 strict
# priority queues selected by the top 3 bits of DSCP / the IPv6 traffic class, which is where Homa writes its
priority.
#
#   h1 eth1 --- eth1 ┐
#   h2 eth1 --- eth2 ├ sw1 (Linux bridge br0; per-port egress: htb rate -> prio 8 bands | single fifo | no
shaping)
#   h3 eth1 --- eth3 │
#   h4 eth1 --- eth4 ┘
LAB_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
LIBVIRT_URI="qemu:///system"

# Base image (tools/build_image.sh): Ubuntu 26.04 cloud image + the mainline 6.17.8 kernel
(kernel.ubuntu.com/mainline, the
# version HomaModule's main branch is developed against) + homa.ko and HomaModule's util/ programs built against
it.
UBUNTU_IMAGE_URL="https://cloud-images.ubuntu.com/releases/resolute/release/ubuntu-26.04-server-cloudimg-
amd64.img"
KERNEL_MAINLINE="v6.17.8"
KERNEL_RELEASE="6.17.8-061708-generic"     # uname -r inside the VMs
HOMA_REPO="https://github.com/PlatformLab/HomaModule.git"
HOMA_COMMIT="65cecd7831e8b24ba7bcd967288bb7ce26fe8eba"   # main, 2026-10-08
BASE_IMAGE="$LAB_DIR/images/homa-base.qcow2"
BASE_DISK_GB=12
BUILDER_NET="default"                     # the image builder VM needs the internet (libvirt NAT)

# Out-of-band management network (isolated libvirt bridge; the host is the gateway). eth0 on every VM.
OOB_NET="homa-oob"
OOB_GATEWAY="10.8.0.1"

# Data network: one L2 segment through sw1 (Linux bridge br0), MTU 9000 (Homa's recommendation)
DATA_PREFIX4="192.168.80.0/24"
DATA_PREFIX6="fd00:80::/64"
DATA_MTU=9000

HOSTS=(h1 h2 h3 h4)
SWITCH=sw1
ALL_NODES=("${HOSTS[@]}" "$SWITCH")
declare -A ROLE=(         [h1]=host [h2]=host [h3]=host [h4]=host [sw1]=switch )
declare -A MGMT_IP=(      [h1]=10.8.0.11 [h2]=10.8.0.12 [h3]=10.8.0.13 [h4]=10.8.0.14 [sw1]=10.8.0.20 )
declare -A DATA_IP=(      [h1]=192.168.80.11 [h2]=192.168.80.12 [h3]=192.168.80.13 [h4]=192.168.80.14 )
declare -A DATA_IP6=(     [h1]=fd00:80::11 [h2]=fd00:80::12 [h3]=fd00:80::13 [h4]=fd00:80::14 )
declare -A SW_PORT=(      [h1]=1 [h2]=2 [h3]=3 [h4]=4 )          # host -> sw1 port (eth<N>)
declare -A NODE_IDX=(     [h1]=1 [h2]=2 [h3]=3 [h4]=4 [sw1]=5 )
declare -A CONSOLE_PORT=( [h1]=5701 [h2]=5702 [h3]=5703 [h4]=5704 [sw1]=5705 )
declare -A VCPU=(         [h1]=2 [h2]=2 [h3]=2 [h4]=2 [sw1]=2 )
declare -A RAM_MIB=(      [h1]=2048 [h2]=2048 [h3]=2048 [h4]=2048 [sw1]=1024 )
BUILDER_VCPU=4
BUILDER_RAM_MIB=4096
BUILDER_CONSOLE=5709
SW_PORTS=4

MAC_OUI="52:54:00:cb"     # MACs 52:54:00:cb:<idx>:<port> (hex); srv6-core :c6, cat8000v-ipsec :c7, vyos-dmvpn
```

```
:c8, :c9, :ca taken
# Links: one isolated libvirt bridge per host <-> sw1 link (networks/homa-link<port>.xml, no addresses, MTU
9000). Not UDP
# tunnels as in the other labs: QEMU's UDP socket backend keeps the host's default 208 KB receive buffer, and
Homa's bursts
# of 9 KB frames overflowed it (host UdpRcvbufErrors) — losses no counter in the lab could see.

# sw1 egress policy applied at boot (sw-qos): mode prio|fifo|off, the per-port rate (the bottleneck), the queue
size per band
SW_QOS_MODE="prio"
SW_QOS_RATE="1gbit"
SW_QOS_LIMIT=200          # packets per queue (x 9000 bytes)

# Homa on the hosts: loaded at boot (/etc/modules-load.d), homa_prio running (systemd unit homa-prio), these
sysctls set
# (/etc/sysctl.d/90-homa.conf; link_mbps = the bottleneck sw1 enforces, so Homa's pacer and grants size for it —
the default
# assumes 25 Gbit/s), and the standing test servers: homa-echo (:4100), homa-echo6 (:4101), cp-homa (:4000), cp-
tcp (:5000), and
# for latency probes cp-homa-probe (:4010), cp-tcp-probe (:5010)
HOMA_PRIO=1
HOMA_SYSCTL=( "link_mbps=1000" )
HOST_SERVICES=(homa-echo homa-echo6 cp-homa cp-tcp cp-homa-probe cp-tcp-probe)
```

Repository layout

```
lab.conf, lab.sh        source of truth, controller
networks/               homa-oob, homa-link1..4
image/                  into the base image: homa-build.sh (builder VM), sw-qos + unit,
                        homa-prio.service, homa.lua (dissector)
tools/build_image.sh    builder VM → images/homa-base.qcow2
tools/homalab.py        every measurement → JSON
tools/build_walkthrough.py   this document (docs/data/ ← the lab; docs/*.html → PDF)
tests/                  Robot: suites/01-07, resources/, run.sh, setup.sh
tests/results_*/        one folder per test run (not in git unless committed)
webapp/                 the portal (port 8108)
images/ nodes/          build products, VM disks and seeds (not in git)
```